S.No: 1	Exp. Name: ***Linear Search***	**Date: 2026-07-29**

Aim:

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input Format:

- The first line contains an integer n, representing the size of the array.
- The second line contains n space-separated integers – the elements of the array.
- The third line contains the integer key – the element to be searched for.

Output Format:

- If the element is found, print its index.
- If the element is not found, print "Not found".

Source Code:

linearSearch.java

ID: 2503921314921043
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

```
import java.util.Scanner;

public class linearSearch {
        public static int search(int arr[],int key,int n)
        {
                for(int i=0;i<n;i++)
                        if(arr[i]==key)
                        {
                                System.out.println(i);
                                return 0;
                        }
                System.out.println("Not found");
                return 1;
        }
        public static void main(String[] args)
        {
                Scanner s = new Scanner(System.in);
                int n = s.nextInt();
                int[] arr=new int[n];
                for(int i=0;i<n;i++)
                        arr[i]=s.nextInt();
                int key = s.nextInt();
                search(arr,key,n);

        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
6
1 2 3 4 5 6
3
2

Test Case - 2
User Output
5

18945
10
Not found

S.No: 2	Exp. Name: ***Write a Java program to Search an element using Binary Search***	**Date: 2026-08-05**

Aim:

Write a class `BinarySearch` with a **public** method `binarySearch` that takes two parameters an `array` of type `int[]` and a `key` of type `int`. Write a code to search the `key` element within the `array` elements by using binary search technique.

Examples for your understanding:

```
Cmd Args : 10 1 2 3 4 5 4
Search element 4 is found at position : 4
```

```
Cmd Args : 10 8 12 11 9
Search element 9 is not found
```

Note: Please don't change the package name.

Source Code:

```
q36663/BinarySearch.java
```


ID: 2503921314921043
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

```
package q36663;
public class BinarySearch {
        public void binarySearch(int[] array,int key)
        {
                int low =0;
                int high = array.length-2;
                while(low <=high)
                        {
                                int mid = (low+high)/2;
                                if(array[mid] == key)
                                {
                                        System.out.println("Search
element " + key + " is found at position : "+(mid));
                                        return;
                                }
                                else if (array[mid]<key)
                                        low=mid+1;
                                else
                                        high = mid-1;
                        }
                System.out.println("Search element " + key + " is not
found");
        }
}
```

q36663/BinarySearchMain.java

```
package q36663;
public class BinarySearchMain{
                public static void main(String[] args){
                int[] array = new int[args.length];
                int n = args.length-1;

                        for (int i = 0; i < n; i++)
                        {
                      array[i] = Integer.parseInt(args[i]);
                        }
                        int key = Integer.parseInt(args[n]);
                        BinarySearch biSearch = new BinarySearch();
                        biSearch.binarySearch(array, key);
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Search element 4 is found at position : 4

Test Case - 2
User Output
Search element 9 is not found

S.No: 3	Exp. Name: ***Program to Sort elements using Selection Sort - Largest element method***	**Date: 2026-08-05**

Aim:

Write a class **SelectionSortingLargestElement** with a **public** method **selectionSortLargestEle**. The method receives one parameter array of type int. Write a code to sort the array elements by using the selection sort - largest element method.

For example:

```
Cmd Args : 63 83 33 53
33
53
63
83
```

Note:

- Make sure to use the **println()** method and not the **print()** method.
- Please don't change the package name.

Source Code:

q11041/SelectionSortingLargestElement.java

```
package q11041;
public class SelectionSortingLargestElement {
	public void selectionSortLargestEle(int[] array)
	{
		int n = array.length;
		for(int i=n-1;i>0;i--)
			{
				int max=0;
				for(int j=1;j<=i;j++)
					if(array[j]>array[max])
						max=j;
				int temp = array[max];
				array[max]=array[i];
				array[i]=temp;
			}
		for(int i=0;i<n;i++)
			System.out.println(array[i]);
	}
}
```

q11041/SelectionSortingMain.java

```
package q11041;
public class SelectionSortingMain{
                public static void main(String[] args) {
                        int[] array = new int[args.length];
                        for (int i = 0; i < args.length; i++)
                        {
                     array[i] = Integer.parseInt(args[i]);
                        }
                        SelectionSortingLargestElement sLarge = new
SelectionSortingLargestElement();
                        sLarge.selectionSortLargestEle(array);
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
33
53
63
83

Test Case - 2
User Output
13
15
18
20
30

ID: 2503921314921043

PSNA College of Engineering and Technology

ID: 2503921314921043
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

S.No: 4	Exp. Name: ***Program to Sort elements using Insertion Sort***	**Date: 2026-09-13**

Aim:

Write a class **InsertionSorting** with a **public** method insertionSort. The method receives one parameter array of int type. Write a code to sort the array elements using the insertion sort technique.

For example:

```
Cmd Args : 10 23 15 8 5
5
8
10
15
23
```

Note:

- Make sure to use the **println()** method and not the **print()** method.
- Please don't change the package name.

Source Code:

q11040/InsertionSorting.java

```
package q11040;

public class InsertionSorting{
	public void insertionSort(int[]arr){
		for(int i=1;i<arr.length;i++){
			int key=arr[i];
			int j=i-1;

			while(j>=0 && arr[j]>key){
				arr[j+1]=arr[j];
				j--;
			}
			arr[j+1]=key;
		}
		for(int i=0;i<arr.length;i++){
			System.out.println(arr[i]);
		}
	}
}
```

q11040/InsertionSortingMain.java

```
package q11040;
public class InsertionSortingMain{
                public static void main(String[] args) {
                        int[] array = new int[args.length];
                        for (int i = 0; i < args.length; i++)
                        {
                     array[i] = Integer.parseInt(args[i]);
                        }
                        InsertionSorting inSorting = new
InsertionSorting();
                        inSorting.insertionSort(array);
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1
3
4
5
6

Test Case - 2
User Output
1
2
3
4

Test Case - 3
User Output
5
8

S.No: 5	Exp. Name: ***Stack Operations Using Array***	**Date: 2026-09-14**

Aim:

Write a program to implement a stack using **Fixed size arrays.**

Write a class called **ClassFixedArrayStack** with all required operations on the stack.

1. `push()`
2. `pop()`
3. `peek()`
4. `display()`
5. `stackSize()`

Note: Driver main method is already provided to run your test cases.

Source Code:

ClassFixedArrayStack.java

ID: 250392131492104 3
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

```
import java.util.Scanner;
public class ClassFixedArrayStack {
    int[] array;
    int top,size;
    public ClassFixedArrayStack(int capacity) {
        this.size=capacity;
        array=new int[size];
        top=-1;
    }

    /*method to insert element into the stack*/
    public void push(int item) {
                if(isFull()){
                        System.out.println("Stack is full item cannot be
added");
                }else{
                        array[++top]=item;
                        System.out.println("Added element is: " + item);
                }
        }
    /*method to remove element from the stack*/
    public int pop() {
                if(isEmpty()){
                        System.out.println("Stack Underflow");
                        return -1;
                }else{
                        int item=array[top--];
                        System.out.println("Element popped is: "+ item);
                        return item;
                }
        }

    /*method to display stack elements*/
    public void display() {
        if(isEmpty()){
                System.out.println("No elements to display");
        }else{
                for(int i=top;i>=0;i--){
                        System.out.println(array[i]);
                }
        }
    }

    /*method to return peek element*/
    public void peek() {
                if(isEmpty()){
```

```
                        System.out.println("Stack is empty");
                }else{
                        System.out.println("Peek of stack is: " +
array[top]);
                }

    }

    /*method to find if stack is empty or not*/
    public boolean isEmpty() {
                return top==-1;

    }

    /*method to find is full or not*/
    public boolean isFull() {
                return top==size - 1;

    }

     /*method to find stack size*/
    public int stackSize(){
                return top + 1;

    }
 public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        System.out.println("Stack Implementation using fixed size
array");
        System.out.print("Enter the size of the stack: ");
        int capacity=sc.nextInt();
        ClassFixedArrayStack stackObject=new
ClassFixedArrayStack(capacity);
        System.out.println("Enter 1 to push\nEnter 2 to pop\nEnter 3 to
display stack\nEnter 4 to display the peek\nEnter 5 to print size of
stack\nEnter 6 to exit");
        while(true){
                System.out.print("Enter your option: ");
                int choice=sc.nextInt();
                        switch(choice){
                                default:
                                        System.out.println("Enter the
right option");
                                        break;
                                case 1:
                                        System.out.print("Scan element
```

```
to push: ");
                                        int item=sc.nextInt();
                                        stackObject.push(item);
                                //      stackObject.display();
                                        break;
                                case 2:
                                        stackObject.pop();
                                        break;
                                case 3:
                                        System.out.print("Elements in
stack: \n");
                                        stackObject.display();
                                        break;
                                case 4:
                                        stackObject.peek();
                                        break;
                                case 5:
                                        System.out.println("Size of
Stack is: "+stackObject.stackSize());
                                        break;
                                case 6:
                                        return;


                }

        }

    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Stack Implementation using fixed size array
Enter the size of the stack:
3
Enter 1 to push
Enter 2 to pop
Enter 3 to display stack
Enter 4 to display the peek

Enter 5 to print size of stack
Enter 6 to exit
Enter your option:
1
Scan element to push:
100
Added element is: 100
Enter your option:
1
Scan element to push:
200
Added element is: 200
Enter your option:
1
Scan element to push:
300
Added element is: 300
Enter your option:
1
Scan element to push:
400
Stack is full item cannot be added
Enter your option:
3
Elements in stack:
300
200
100
Enter your option:
4
Peek of stack is: 300
Enter your option:
5
Size of Stack is: 3
Enter your option:
2
Element popped is: 300
Enter your option:
2
Element popped is: 200
Enter your option:

2
Element popped is: 100
Enter your option:
2
Stack Underflow
Enter your option:
3
Elements in stack:
No elements to display
Enter your option:
4
Stack is empty
Enter your option:
5
Size of Stack is: 0
Enter your option:
6

S.No: 6	Exp. Name: ***Implementation of the linear queue***	**Date: 2026-10-19**

Aim:

Write a Java Program to Implement the linear queue

Complete the methods public boolean **isFull()**, public boolean **isEmpty()**, public void **enqueue(int item)**, public void **dequeue(),** and public void **display()** in the given editor.

Note: The driver code is provided to you in the editor.

Source Code:

```
q36649/Linearqueue.java
```


ID: 250392131492104 3
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

```
package q36649;
import java.util.*;
class Linearqueue {
  int maxCapacity;
  int items[];
  int front;
  int rear;

  public Linearqueue(int size) {
        front = -1;
        rear = -1;
        this.maxCapacity = size;
        items = new int[maxCapacity];
  }

public boolean isFull() {
        return rear == maxCapacity - 1;
  }

public boolean isEmpty() {
        return front == -1 || front > rear;
  }

public void enqueue(int item) {
        if (isFull()) {
                System.out.println("Queue status is Overflow");
                return;
        }
        if (front == -1) {
                front = 0;
        }
        rear++;
        items[rear] = item;
        System.out.println("Item Inserted: " + item);
  }

  public void dequeue() {
        if (isEmpty()) {
                System.out.println("Queue is empty");
                return;
        }
        System.out.println("Element deleted: "  + items[front]);
        front++;
        if (front > rear) {
                front = -1;
                rear = -1;
```

```
        }
  }

/* Method to display elements of Queue */
  public void display() {
        if (isEmpty()) {
                System.out.println("Queue is empty");
                return;
        }

        for (int i = front; i <= rear; i++) {
                System.out.print(items[i] + "  ");
        }
          System.out.println();
  }


/* Driver main method */
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.println("Queue Implmentation using fixed size array");
    System.out.print("Enter the size of queue: ");
    int capacity = sc.nextInt();
    int item;
    Linearqueue queue = new Linearqueue(capacity);
    System.out.println("Enter 1 to Insert\nEnter 2 to Delete\nEnter 3 to
Display\nEnter 4 to Exit");
    while(true) {
        System.out.print("Enter your option: ");
        int choice = sc.nextInt();
        switch(choice){
                default:
                        System.out.println("Enter the right option");
                        break;
                case 1:
                        System.out.print("Enter element to insert: ");
                        item = sc.nextInt();
                        queue.enqueue(item);
                        break;
                case 2:
                        queue.dequeue();
                        break;
                case 3:
                        System.out.println("Elements in queue are:");
                        queue.display();
                                        //System.out.println("\n");
```

```
                        break;
                case 4:
                        System.exit(1);

        }

    }

  }

}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Queue Implmentation using fixed size array
Enter the size of queue:
4
Enter 1 to Insert
Enter 2 to Delete
Enter 3 to Display
Enter 4 to Exit
Enter your option:
1
Enter element to insert:
4
Item Inserted: 4
Enter your option:
1
Enter element to insert:
7
Item Inserted: 7
Enter your option:
1
Enter element to insert:
9
Item Inserted: 9

3
Elements in queue are:
30
Enter your option:
2
Element deleted: 30
Enter your option:
3
Elements in queue are:
Queue is empty
Enter your option:
2
Queue is empty
Enter your option:
4

S.No: 7	Exp. Name: ***Generate Pay Slips for Employees***	**Date: 2026-10-19**

Aim:

Develop a Java application with an **Employee** class with **Emp_name, Emp_id, Address, Mail_id, Mobile_no** as members. Inherit the classes, Programmer, Assistant Professor, Associate Professor and Professor from employee class. Add Basic Pay (BP) as the member of all the inherited classes with 97% of BP as DA, 10 % of BP as HRA, 12% of BP as PF, and 0.1% of BP for staff club fund. Generate pay slips for the employees with their gross and net salary.

Input Format:

The program will prompt the user to input details for four types of employees (Programmer, Assistant Professor, Associate Professor, and Professor). For each employee, the following details are required:

- Employee Name: The name of the employee.
- Employee ID: A unique ID for the employee.
- Address: The employee's address.
- Mail ID: The email address of the employee.
- Mobile Number: The mobile number of the employee (long integer).
- Basic Pay: The employee's basic pay (a floating-point number).

Each employee's details will be entered one by one. After the details for each employee are entered, the payslip will be generated.

Output Format:

After entering the details, the program will generate and display a pay slip for each employee. The output will show:

- Employee Name: Name of the employee.
- Employee ID: The unique ID of the employee.
- Address: The address of the employee.
- Mail ID: The email of the employee.
- Mobile Number: The mobile number of the employee.
- Basic Pay: The basic pay entered for the employee.
- DA (Dearness Allowance): Calculated as 97% of the Basic Pay.
- HRA (House Rent Allowance): Calculated as 10% of the Basic Pay.
- PF (Provident Fund): Calculated as 12% of the Basic Pay.
- Staff Club Fund: Calculated as 0.1% of the Basic Pay.
- Gross Salary: Sum of Basic Pay + DA + HRA.
- Net Salary: Gross Salary - PF - Staff Club Fund.

Note:

- Refer to visible test cases for better understanding.

Source Code:

```
import java.util.Scanner;
class Employee{
        String empName;
        int empId;
        String address;
        String mailId;
        String mobileNo;
        Employee(String empName,int empId,String address,String
mailId,String mobileNo){
                this.empName = empName;
                this.empId = empId;
                this.address = address;
                this.mailId = mailId;
                this.mobileNo = mobileNo;
        }
        void displayDetails(){
                System.out.println("Employee Name: " + empName);
                System.out.println("Employee ID: " + empId);
                System.out.println("Address: " + address);
                System.out.println("Mail ID: " + mailId);
                System.out.println("Mobile Number: " + mobileNo);
        }
        void displayPaySlip(double basicPay){
                double da = basicPay * 0.97;
                double hra = basicPay * 0.10;
                double pf = basicPay * 0.12;
                double staffClubFund = basicPay * 0.001;
                double grossSalary = basicPay + da + hra;
                double netSalary = grossSalary - pf - staffClubFund;

                System.out.println("Pay Slip for " + empName);
                System.out.println("Employee ID: " + empId);
                System.out.println("Address: " + address);
                System.out.println("Mail ID: " + mailId);
                System.out.println("Mobile Number: " + mobileNo);

                System.out.printf("Basic Pay: %.1f%n",basicPay);
                System.out.printf("DA: %.1f%n",da);
                System.out.printf("HRA: %.1f%n",hra);
                System.out.printf("PF: %.1f%n",pf);
                System.out.printf("Staff Club Fund:
%.1f%n",staffClubFund);
                System.out.printf("Gross Salary: %.1f%n",grossSalary);
                System.out.printf("Net Salary: %.1f%n",netSalary);
        }
};
```

```
class Programmer extends Employee {
        double basicPay;
        Programmer(String empName,int empId,String address,String
mailId,String mobileNo,double basicPay){
                super(empName,empId,address,mailId,mobileNo);
                this.basicPay = basicPay;
        }
        void generatePaySlip(){

                displayPaySlip(basicPay);
        }
};

class AssistantProfessor extends Employee{
        double basicPay;
        AssistantProfessor(String empName,int empId,String
address,String mailId,String mobileNo,double basicPay){
                super(empName,empId,address,mailId,mobileNo);
                this.basicPay = basicPay;
        }
                void generatePaySlip(){

                        displayPaySlip(basicPay);
        }
};
class AssociateProfessor extends Employee{
        double basicPay;
        AssociateProfessor(String empName,int empId,String
address,String mailId,String mobileNo,double basicPay){
                super(empName,empId,address,mailId,mobileNo);
                this.basicPay = basicPay;
        }
        void generatePaySlip(){

                displayPaySlip(basicPay);
        }
};
class Professor extends Employee{
        double basicPay;
        Professor(String empName,int empId,String address,String
mailId,String mobileNo,double basicPay){
                super(empName,empId,address,mailId,mobileNo);
                this.basicPay = basicPay;
        }
        void generatePaySlip(){
                displayPaySlip(basicPay);
```

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 28

```
        }
};
                public class SalaryManagementSystem{
                        public static void main(String[] args){
                                Scanner sc = new Scanner(System.in);

                                System.out.println("Enter details for
Programmer:");
                                System.out.print("Employee Name: ");
                                String n1 = sc.nextLine();
                                System.out.print("Employee ID: ");
                                int id1 =
Integer.parseInt(sc.nextLine());
                                System.out.print("Address: ");
                                String a1 = sc.nextLine();
                                System.out.print("Mail ID: ");
                                String m1 = sc.nextLine();
                                System.out.print("Mobile Number: ");
                                String mo1=sc.nextLine();
                                System.out.print("Basic Pay: ");
                                double b1 =
Double.parseDouble(sc.nextLine());
                                Programmer p1 = new
Programmer(n1,id1,a1,m1,mo1,b1);
                                p1.generatePaySlip();
                                System.out.println("Enter details for
Assistant Professor:");
                                System.out.print("Employee Name: ");
                                String n2 = sc.nextLine();
                                System.out.print("Employee ID: ");
                                int id2 =
Integer.parseInt(sc.nextLine());
                                System.out.print("Address: ");
                                String a2 = sc.nextLine();
                                System.out.print("Mail ID: ");
                                String m2 = sc.nextLine();
                                System.out.print("Mobile Number: ");
                                String mo2=sc.nextLine();
                                System.out.print("Basic Pay: ");
                                double b2 =
Double.parseDouble(sc.nextLine());
                                AssistantProfessor p2 = new
AssistantProfessor(n2,id2,a2,m2,mo2,b2);
                                p2.generatePaySlip();
                                System.out.println("Enter details for
Associate Professor:");
```

```
                                System.out.print("Employee Name: ");
                                String n3 = sc.nextLine();
                                System.out.print("Employee ID: ");
                                int id3 =
Integer.parseInt(sc.nextLine());
                                System.out.print("Address: ");
                                String a3 = sc.nextLine();
                                System.out.print("Mail ID: ");
                                String m3 = sc.nextLine();
                                System.out.print("Mobile Number: ");
                                String mo3=sc.nextLine();
                                System.out.print("Basic Pay: ");
                                double b3 =
Double.parseDouble(sc.nextLine());
                                AssociateProfessor p3 = new
AssociateProfessor(n3,id3,a3,m3,mo3,b3);
                                p3.generatePaySlip();
                                System.out.println("Enter details for
Professor:");
                                System.out.print("Employee Name: ");
                                String n4 = sc.nextLine();
                                System.out.print("Employee ID: ");
                                int id4 =
Integer.parseInt(sc.nextLine());
                                System.out.print("Address: ");
                                String a4 = sc.nextLine();
                                System.out.print("Mail ID: ");
                                String m4 = sc.nextLine();
                                System.out.print("Mobile Number: ");
                                String mo4=sc.nextLine();
                                System.out.print("Basic Pay: ");
                                double b4 =
Double.parseDouble(sc.nextLine());
                                Professor p4 = new
Professor(n4,id4,a4,m4,mo4,b4);
                                p4.generatePaySlip();

                        }
                }
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter details for Programmer:
Employee Name:
John
Employee ID:
100
Address:
Princeton
Mail ID:
j23@gmail.com
Mobile Number:
9524304564
Basic Pay:
10000
Pay Slip for John
Employee ID: 100
Address: Princeton
Mail ID: j23@gmail.com
Mobile Number: 9524304564
Basic Pay: 10000.0
DA: 9700.0
HRA: 1000.0
PF: 1200.0
Staff Club Fund: 10.0
Gross Salary: 20700.0
Net Salary: 19490.0
Enter details for Assistant Professor:
Employee Name:
Clarie
Employee ID:
101
Address:
Ottawa
Mail ID:
cl34@gmail.com
Mobile Number:
8500212274

Employee ID:
105
Address:
Loss angels
Mail ID:
krim@gmail.com
Mobile Number:
9010692094
Basic Pay:
15000
Pay Slip for Krem
Employee ID: 105
Address: Loss angels
Mail ID: krim@gmail.com
Mobile Number: 9010692094
Basic Pay: 15000.0
DA: 14550.0
HRA: 1500.0
PF: 1800.0
Staff Club Fund: 15.0
Gross Salary: 31050.0
Net Salary: 29235.0

S.No: 8	Exp. Name: ***Area of the Shape***	**Date: 2026-09-13**

Aim:

Write a Java Program to create an abstract class named **Shape** that contains two integers and an empty method named **printArea()**. Provide three classes named **Rectangle**, **Triangle,** and **Circle** such that each one of the classes extends the class **Shape**. Each one of the classes contains only the method **printArea ()** that prints the area of the given shape. Prompt the user to enter the dimensions like length l, width w, base b, height h, and radius r of each shape as required.

Input Format:

- The first two lines of the input consist of the length, width of the rectangle (integer).
- The next two lines of the input consist of the base, height of the triangle (integer).
- The next line of the input consists of the radius of the circle (integer).

Output Format:

- The output represents the area (float) of each shape in the next line after taking dimensions.

Constraint:

- $1 \leq l \leq 100$
- $1 \leq w \leq 100$
- $1 \leq b \leq 100$
- $1 \leq h \leq 100$
- $1 \leq r \leq 100$

Note:

- For the value of π, utilize the **Math.PI** constant and the driver code is provided in the editor.

Source Code:

```
q17184/ShapeTest.java
```

Page No: 34
ID: 2503921314921043
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

```
package q17184;
import java.util.Scanner;

abstract class Shape {
        int val1;
        int val2;

        abstract void printArea();
}
class Rectangle extends Shape {
        Rectangle(int length,int width){
                this.val1=length;
                this.val2=width;
        }
        void printArea() {
                System.out.println("Area of Rectangle: "+(val1*val2));
        }
}
class Triangle extends Shape {
        Triangle(int base,int height){
                this.val1=base;
                this.val2=height;
        }
        void printArea(){
                double area=0.5*val1*val2;
                    System.out.println("Area of Triangle: "+area);
        }
}

class Circle extends Shape{
        Circle(int radius){
                this.val1=radius;
        }
        void printArea(){
                double area=Math.PI*val1*val1;
                System.out.println("Area of Circle: "+area);
        }
}
public class ShapeTest {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.println("Enter dimensions for Rectangle:");
        System.out.print("Length: ");
        int length = input.nextInt();
        System.out.print("Width: ");
```

```
        int width = input.nextInt();
        Rectangle rectangle = new Rectangle(length, width);
        rectangle.printArea();

        System.out.println("Enter dimensions for Triangle:");
        System.out.print("Base: ");
        int base = input.nextInt();
        System.out.print("Height: ");
        int height = input.nextInt();
        Triangle triangle = new Triangle(base, height);
        triangle.printArea();

        System.out.println("Enter dimensions for Circle:");
        System.out.print("Radius: ");
        int radius = input.nextInt();
        Circle circle = new Circle(radius);
        circle.printArea();

        input.close();
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter dimensions for Rectangle:
Length:
10
Width:
20
Area of Rectangle: 200
Enter dimensions for Triangle:
Base:
20
Height:
20
Area of Triangle: 200.0
Enter dimensions for Circle:
Radius:

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 36

21
Area of Circle: 1385.442360233099

Test Case - 2
User Output
Enter dimensions for Rectangle:
Length:
11
Width:
12
Area of Rectangle: 132
Enter dimensions for Triangle:
Base:
13
Height:
14
Area of Triangle: 91.0
Enter dimensions for Circle:
Radius:
15
Area of Circle: 706.8583470577034

S.No: 9	Exp. Name: ***Area of Triangle, Rectangle and Circle using Interface***	**Date: 2026-09-13**

Aim:

Define an interface **GeometricShape** with methods **area()** and **perimeter()**. Both methods return type and parameter list should be void and empty, respectively.

Create the following classes that implement the **GeometricShape** interface:

- **Triangle**
- **Rectangle**
- **Circle**

Each class should have:

- Have the required member variables to store dimensions:
  - **Triangle**: side1, side2, side3
  - **Rectangle**: length, breadth**Circle**: radius
- Implement the **area()** and **perimeter()** methods:

For the triangle, use Heron's formula:

```
s = (side1 + side2 + side3) / 2
area = sqrt(s * (s - side1) * (s - side2) * (s - side3))
Perimeter = side1 + side2 + side3
```

Print:

```
Area of Triangle:<value>
Perimeter of Triangle:<value>
```

For the rectangle:

```
Area = length * breadth
Perimeter = 2 * (length + breadth)
```

Print:

```
Area of Rectangle:<value>
Perimeter of Rectangle:<value>
```

For the circle:

```
Area = Math.PI * radius * radius
Circumference = 2 * Math.PI * radius
```

Print:

```
Area of Circle:<value>
Circumference of Circle:<value>
```

- The main method will handle user input and call the **area()** and **perimeter()** methods. The `main` method is provided to you in the editor.

Your task is to complete the **GeometricShape** interface and the required classes so that the program works as expected.

Input Format:

- The first line of the input consists of a number representing the choice to select a shape (integer).
- The next line of the input consists of the dimensions of the shape, respectively (integer).

Output Format:

- The first line of the output represents the area of the selected shape (`float`).
- The second line of the output represents the perimeter of the selected shape (`float`).
- If an invalid choice is entered, it prints: "Invalid choice".

Note:

- Use `Math.sqrt()` for square root and `Math.PI` for the value of π.
- Refer to the visible test cases to strictly match the input/output layout

Source Code:

```
q17207/ExecuteMain.java
```

```
package q17207;
import java.util.Scanner;

interface GeometricShape {
        void area();
        void perimeter();
}

class Triangle implements GeometricShape {
        double side1, side2, side3;

        public void area() {
                double s = (side1 + side2 + side3) / 2;
                double area = Math.sqrt(s * (s - side1) * (s - side2) *
(s - side3));

                System.out.println("Area of Triangle:" + area);
        }

        public void perimeter() {
                double perimeter = side1 + side2 + side3;

                System.out.println("Perimeter of Triangle:" +
perimeter);
        }
}

class Rectangle implements GeometricShape {
        double length, breadth;
        public void area() {
                double area = length * breadth;

                System.out.println("Area of Rectangle:" + area);
        }

        public void perimeter() {
                double perimeter = 2 * (length + breadth);
                System.out.println("Perimeter of Rectangle:" +
perimeter);
        }
}
class Circle implements GeometricShape {
        double radius;

        public void area(){
                double area = Math.PI * radius * radius;
```

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 40

```
                System.out.println("Area of Circle:" + area);
        }

        public void perimeter() {
        double circumference = 2 * Math.PI * radius;

        System.out.println("Circumference of Circle:" + circumference);

        }
}

// Main class for testing
public class ExecuteMain {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Choose a geometric shape:");
        System.out.println("1.Triangle");
        System.out.println("2.Rectangle");
        System.out.println("3.Circle");
        System.out.println("Enter your choice:");

        int choice = scanner.nextInt();
        GeometricShape shape;

        switch (choice) {
            case 1:
                shape = new Triangle();
                break;
            case 2:
                shape = new Rectangle();
                break;
            case 3:
                shape = new Circle();
                break;
            default:
                System.out.println("Invalid choice");
                scanner.close();
                return;
        }

        if (shape instanceof Triangle) {
            System.out.println("Enter lengths of the three sides of
Triangle:");
            ((Triangle) shape).side1 = scanner.nextDouble();
```

```
            ((Triangle) shape).side2 = scanner.nextDouble();
            ((Triangle) shape).side3 = scanner.nextDouble();
        } else if (shape instanceof Rectangle) {
            System.out.println("Enter length and breadth of
Rectangle:");
            ((Rectangle) shape).length = scanner.nextDouble();
            ((Rectangle) shape).breadth = scanner.nextDouble();
        } else if (shape instanceof Circle) {
            System.out.println("Enter radius of Circle:");
            ((Circle) shape).radius = scanner.nextDouble();
        }

        shape.area();
        shape.perimeter();

        scanner.close();
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Choose a geometric shape:
1.Triangle
2.Rectangle
3.Circle
Enter your choice:
1
Enter lengths of the three sides of Triangle:
3 4 5
Area of Triangle:6.0
Perimeter of Triangle:12.0

Test Case - 2
User Output
Choose a geometric shape:
1.Triangle
2.Rectangle

3.Circle
Enter your choice:
2
Enter length and breadth of Rectangle:
10 20
Area of Rectangle:200.0
Perimeter of Rectangle:60.0

Test Case - 3
User Output
Choose a geometric shape:
1.Triangle
2.Rectangle
3.Circle
Enter your choice:
3
Enter radius of Circle:
6
Area of Circle:113.09733552923255
Circumference of Circle:37.69911184307752

Test Case - 4
User Output
Choose a geometric shape:
1.Triangle
2.Rectangle
3.Circle
Enter your choice:
5
Invalid choice

S.No: 10	Exp. Name: ***Write the code to implement exception handling.***	**Date: 2026-10-19**

Aim:

Write Java program that uses the exception handling features of the language, creates exceptions and handles them properly, uses the predefined exceptions, and create own exceptions. handle all the exceptions using try and catch block. The following Java code provides a menu-driven interface to demonstrate different types of exception handling. Interact with the program by choosing options from the menu and providing input as prompted to observe how various exceptions are handled.

Input Format:

The program first prompts you to choose an option from the following menu by entering a number:

```
1.ArithmeticException
2.ArrayIndexOutOfBoundsException
3.NumberFormatException
4.Custom Exception
5.Exit
```

Based on your choice, the program will then prompt you for further input:

- **ArithmeticException (Option 1):** You will be asked to enter an integer for the numerator and then an integer for the denominator.
- **ArrayIndexOutOfBoundsException (Option 2):** You will be asked to enter an integer for the array size, and then an integer for the index to access.
- **NumberFormatException (Option 3):** You will be asked to enter a numeric string.
- **Custom Exception (Option 4):** You will be asked to enter an integer for the age.
- **Exit (Option 5):** No further input is required.

Output Format:

The program will first display the menu of exception types. After you make a selection and provide the necessary input, the output will depend on whether an exception is triggered and how it is handled. The output will generally follow one of these formats:

- **Successful Operation (No Exception):** If the input does not cause an exception, the program will display the result of the operation (e.g., the calculated result of division, the value at the accessed array index, or the entered number/age).
- **Caught Exception:** If the input causes a specific exception, the program will catch it and print an error message indicating the type of exception and a brief description (obtained using the **getMessage**() method of the exception object). For the custom exception, the message defined in the exception class will be displayed.
- **Invalid Input (for menu choice or subsequent input):** If you enter non-integer input when an integer is expected, an "Invalid input" message will be displayed, and you will be prompted again.

The menu will continue to be displayed until you choose option 5 to exit the program.

ID: 250392131492104 3
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

```
import java.util.InputMismatchException;
import java.util.Scanner;

public class ExceptionHandlingDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int option;

        do {
            System.out.println("Choose an option to explore
exceptions:");
            System.out.println("1. ArithmeticException");
            System.out.println("2. ArrayIndexOutOfBoundsException");
            System.out.println("3. NumberFormatException");
            System.out.println("4. Custom Exception");
            System.out.println("5. Exit");

            option = scanner.nextInt();

            switch (option) {
                case 1:
                    exploreArithmeticException();
                    break;
                case 2:
                    exploreArrayIndexOutOfBoundsException();
                    break;
                case 3:
                    exploreNumberFormatException();
                    break;
                case 4:
                    exploreCustomException();
                    break;
                case 5:
                    break;
                default:
                    System.out.println("Invalid selection");
            }
        } while (option != 5);
    }

//write your code here for the given exceptions...
static class InvalidAgeException extends Exception {
        public InvalidAgeException(String msg){
                super(msg);
        }
}
```

```
        public static void exploreArithmeticException(){
                Scanner sc = new Scanner(System.in);
                System.out.print("Numerator: ");
                int num = sc.nextInt();
                System.out.print("Denominator: ");
                int den = sc.nextInt();
                try {
                        int res = num/den;
                        System.out.println("Result: " + res);
                }catch(ArithmeticException e){
                        System.out.println("ArithmeticException: " +
e.getMessage());
                }
        }
        public static void exploreArrayIndexOutOfBoundsException(){
                Scanner sc = new Scanner(System.in);
                System.out.print("Array size: ");
                int size = sc.nextInt();
                System.out.print("Enter an index to access in the array:
");
                int idx = sc.nextInt();
                try{
                        int[] arr=new int[size];
                        int v = arr[idx];
                        System.out.println("Value at index " + idx + ":
" + v);
                }catch (ArrayIndexOutOfBoundsException e){

System.out.println("ArrayIndexOutOfBoundsException: " + e.getMessage());
                }
        }
        public static void exploreNumberFormatException(){
                Scanner sc = new Scanner(System.in);
                System.out.print("Numeric string: ");
                String s = sc.next();
                try{
                        int n = Integer.parseInt(s);
                        System.out.println("Number: " + n);
                }catch (NumberFormatException e){
                        System.out.println("NumberFormatException: " +
e.getMessage());
                }
        }
        public static void exploreCustomException(){
                Scanner sc = new Scanner(System.in);
                System.out.print("Age: ");
```

PSNA College of Engineering and Technology 2025-2029-CSE(CS) ID: 250392131492104 3 Page No: 47

```
                int age = sc.nextInt();
                try {
                        if(age<0){
                                throw new InvalidAgeException("Age
cannot be negative");
                        }else{
                                System.out.println("Age: "+age);
                        }
                }catch (InvalidAgeException e){
                        System.out.println("Custom Exception: " +
e.getMessage());
                }
        }



}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Choose an option to explore exceptions:
1. ArithmeticException
2. ArrayIndexOutOfBoundsException
3. NumberFormatException
4. Custom Exception
5. Exit
1
Numerator:
10
Denominator:
0
ArithmeticException: / by zero
Choose an option to explore exceptions:
1. ArithmeticException
2. ArrayIndexOutOfBoundsException
3. NumberFormatException

4. Custom Exception
5. Exit
2
Array size:
5
Enter an index to access in the array:
9
ArrayIndexOutOfBoundsException: Index 9 out of bounds for length 5
Choose an option to explore exceptions:
1. ArithmeticException
2. ArrayIndexOutOfBoundsException
3. NumberFormatException
4. Custom Exception
5. Exit
3
Numeric string:
java
NumberFormatException: For input string: "java"
Choose an option to explore exceptions:
1. ArithmeticException
2. ArrayIndexOutOfBoundsException
3. NumberFormatException
4. Custom Exception
5. Exit
4
Age:
-22
Custom Exception: Age cannot be negative
Choose an option to explore exceptions:
1. ArithmeticException
2. ArrayIndexOutOfBoundsException
3. NumberFormatException
4. Custom Exception
5. Exit
5

Test Case - 2
User Output
Choose an option to explore exceptions:
1. ArithmeticException

Age: 45
Choose an option to explore exceptions:
1. ArithmeticException
2. ArrayIndexOutOfBoundsException
3. NumberFormatException
4. Custom Exception
5. Exit
4
Age:
-98
Custom Exception: Age cannot be negative
Choose an option to explore exceptions:
1. ArithmeticException
2. ArrayIndexOutOfBoundsException
3. NumberFormatException
4. Custom Exception
5. Exit
5

S.No: 11	Exp. Name: ***Multi Threaded Application with Three Threads***	**Date: 2026-09-14**

ID: 2503921314921043
2025-2029-CSE(CS)
PSNA College of Engineering and Technology

Aim:

Write a Java program that demonstrates multithreading where:

1. A **RandomNumberThread** generates 3 random integers (seeded with 50)
2. For each generated number:
3. If the number is even, create a **SquareThread** to calculate its square.
4. If the number is odd, create a **CubeThread** to calculate its cube.
5. Each operation should be printed with a 1-second delay between numbers.

Input Format:

- No direct input is required. The random number generator is seeded with 50 (fixed sequence).

Output Format:

```
Random Integer generated : <number1>
Square/Cube of <number1> = <result1>
[1-second delay]
Random Integer generated : <number2>
Square/Cube of <number2> = <result2>
[1-second delay]
Random Integer generated : <number3>
Square/Cube of <number3> = <result3>
```

Example Output:

```
Random Integer generated : 42
Square of 42 = 1764
Random Integer generated : 29
Cube of 29 = 24389
Random Integer generated : 74
Square of 74 = 5476
```

Constraints:

- The RandomNumberThread must generate exactly 3 numbers.
- Use Thread.sleep(1000) for the 1-second delay.
- Handle thread creation and synchronization implicitly (no explicit synchronization needed).

Note:

- Set the seed value to 50 for consistency.
- The print statements for the output are already present in the editor. Your task is to implement the logic in the specified sections of the code.
- Refer to the visible test cases to strictly match the input/output layout.

Source Code:

```
import java.util.Random;
class RandomNumberThread extends Thread {
        public void run() {
                Random random = new Random();
                random.setSeed(50);

                for (int i = 0; i < 3; i++) {
                        int num = random.nextInt(100);
                        System.out.println("Random Integer generated : "
+ num);

                        if(num % 2 == 0) {
                                SquareThread st = new SquareThread(num);
                                st.start();
                                try{
                                        st.join();
                                } catch(InterruptedException e) {
                                        e.printStackTrace();
                                }
                        }else {
                                CubeThread ct = new CubeThread(num);
                                ct.start();
                                try{
                                        ct.join();
                                }catch (InterruptedException e) {
                                        e.printStackTrace();
                                }
                        }
                }
        }
}

class SquareThread extends Thread {
        int number;

        SquareThread(int number) {
                this.number = number;
        }
        public void run() {
                System.out.println("Square of " + number + " = " +
(number * number));
        }
}

class CubeThread extends Thread {
        int number;
```

```
        CubeThread(int number) {
                this.number = number;
        }
        public void run(){
                System.out.println("Cube of " + number + " = " + (number
* number * number));
        }
}

public class ClassMthread {
        public static void main(String[] args) {
                RandomNumberThread rnt = new RandomNumberThread();
                rnt.start();
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Random Integer generated : 17
Cube of 17 = 4913
Random Integer generated : 88
Square of 88 = 7744
Random Integer generated : 93
Cube of 93 = 804357

S.No: 12	Exp. Name: ***Find and Replace in a File***	**Date: 2026-09-14**

Aim:

Write a Java program to find and replace occurrences of a given search pattern with a specified replacement string within a given file. The program should then read and print the content of the modified file to the console.

Input Format:

- The program will first read the path to the input file: <file name>.
- Then, it will read the string to be searched for: $old - string$.
- Finally, it will read the replacement string: $new - string$.

Output Format:

- The program will first print the label: "Modified File Content:", followed by each line of the modified file on a new line.

Note:

- Use file names as sample1.txt and sample2.txt.
- Observe the visible test cases for better understanding.

Source Code:

```
FindReplaceAndReadFile.java
```

```
import java.io.*;
import java.util.Scanner;

public class FindReplaceAndReadFile {
    public static void main(String[] args) {
                Scanner scanner = new Scanner(System.in);

                String fileName = scanner.nextLine();
                String oldString = scanner.nextLine();
        String newString = scanner.nextLine();
                try {
                        File file = new File(fileName);
                        Scanner fileScanner = new Scanner(file);

                        System.out.println("Modified File Content:");
                        while (fileScanner.hasNextLine()) {
                                String line = fileScanner.nextLine();
                                String modifiedLine =
line.replace(oldString, newString);
                                System.out.println(modifiedLine);
                        }
                        fileScanner.close();
                }
                catch(IOException e){
            e.printStackTrace();
        }
    }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
sample1.txt
codetangtra
CodeTantra
Modified File Content:
I love coding with CodeTantra

Test Case - 2

User Output
sample2.txt
dancing
coding
Modified File Content:
You are goot at coding